

Desafio Bioinformático: Predição de ncRNAs em *Leishmania braziliensis*

Pipeline Tryp-ncRNA e Análise Downstream Integrada

Wasim A. P. Syed

2026-08-05

Contents

Observação	1
Pacotes	2
Preparar arquivos input	2
Definir diretórios	2
Baixar genoma	3
Baixar reads via ENA	3
Quebrar sequencia para rodar mais rápido	5
Analisar qualidade do FastQC	5
Predição via TrypncRNA	13
Baixar repositório	13
Rodar no Docker	14
Analisar resultados	15

Observação

Devido a limitações computacionais locais na execução do pipeline (mesmo em ambiente isolado via Conda/Docker e com subamostragem dos arquivos FASTQ) e à ausência de dados de exemplo no repositório original, optou-se pela utilização de um conjunto de dados público de ncRNAs de *Leishmania* baixado do artigo intitulado “Comparative and systems analyses of *Leishmania* spp. non-coding RNAs through developmental stages” (PLOS Neglected Tropical Diseases, DOI: 10.1371/journal.pntd.0013108), permitindo o desenvolvimento e a validação do script de visualização dos resultados.

O presente Markdown contém os scripts para preparar os dados utilizados como input no pipeline TriTryp-ncRNA, bem como a execução do mesmo adaptado para o ambiente R/Markdown. Além disso, contém também os códigos para limpeza do dataset do artigo citado acima e para a visualização simples desses dados.

Pacotes

```
# Baixar pacotes do R =====
# install.packages(c("here", "fs", "gert", "rentrez", "reticulate", "BiocManager", "tidyverse"))
# BiocManager::install("fastqcr")
# BiocManager::install("ShortRead")
# Carregar pacotes
library(here)
library(fs)
library(gert)
library(BiocManager)
library(BiocFileCache)
library(rentrez)
library(reticulate)
library(tidyverse)
library(fastqcr)
library(ShortRead)
library(scales)
library(ggsci)
library(ggdists)
library(patchwork)

## Setup no terminal =====

# sudo apt update
# #Install miniconda
# wget https://repo.continuum.io/miniconda/Miniconda3-latest-Linux-x86_64.sh
# bash Miniconda3-latest-Linux-x86_64.sh
# #Install Python
# sudo apt install -y python3-venv python3-pip
#
# #Instalar pacotes necessários
# #Instalando o SRA Toolkit (rodar no terminal)
# sudo apt install sra-toolkit
# fasterq-dump --version
#
# # 1. Dar permissão de execução no arquivo dentro da pasta atual
# chmod +x ./TriTry-ncRNA/scripts/instalar_dependencias.sh
#
# # 2. Executar o script
# ./TriTry-ncRNA/scripts/instalar_dependencias.sh
```

Preparar arquivos input

Definir diretórios

```
# Definir diretórios
project_dir = here::here("Colaborações", "Angela Kaysel")
data_dir <- here::here(project_dir, "data")
```

```
# Define sample accession and workspace directory paths
target_srr <- "SRR7945375"
sample_dir <- fs::path(data_dir, target_srr)
qc_dir      <- fs::path(sample_dir, "fastqc_results")

# Make directories
fs::dir_create(data_dir, sample_dir, qc_dir)
```

Baixar genoma

Já existe no TriTryp ncRNA, mas se fosse necessário, poderia rodar o seguinte código.

```
# # Download do Genoma e Anotação de L. braziliensis =====
# # Evita re-baixar arquivos se já existirem
# bfc <- BiocFileCache(cache = here::here(".bioc_cache"), ask = FALSE)
#
# #Baixar arquivos do NCBI via FTP
# ncbi_base_url <- "https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/000/002/725/GCF_000002725.2_ASM272v2"
# #Genoma
# genome_url    <- paste0(ncbi_base_url, "/GCF_000002725.2_ASM272v2_genomic.fna.gz")
# #Anotação do genoma
# gff_url       <- paste0(ncbi_base_url, "/GCF_000002725.2_ASM272v2_genomic.gff.gz")
#
# # Baixar
# genome_file <- bfcpath(bfc, genome_url)
# gff_file    <- bfcpath(bfc, gff_url)
#
# # Copiar/criar links para a pasta data/ com nomes padronizados
# fs::file_copy(genome_file, fs::path(data_dir, "Lbraziliensis_Genome.fasta"),
#               overwrite = TRUE)
# fs::file_copy(gff_file, fs::path(data_dir, "Lbraziliensis_Annotation.gff"),
#               overwrite = TRUE)
```

Baixar reads via ENA

Construir query e baixar

```
# Construct ENA Portal API query URL
ena_api_url <- sprintf(
  "https://www.ebi.ac.uk/ena/portal/api/filereport?accession=%s&result=read_run&fields=fastq ftp&format=fastq"
  target_srr
)

# Fetch ENA metadata
ena_info <- readr::read_tsv(ena_api_url, show_col_types = FALSE)

fastq_urls <- ena_info %>%
  dplyr::pull(fastq ftp) %>%
  stringr::str_split(";") %>%
  unlist() %>%
  paste0("https://", .)
```

```

# Download paired FASTQ files directly
options(timeout = max(3600, getOption("timeout")))

purrr::walk(fastq_urls, function(url) {
  file_name <- basename(url)
  dest_file <- fs::path(sample_dir, file_name)

  # Download compressed FASTQ file if not present in the local sample directory
  if (!fs::file_exists(dest_file)) {
    message(stringr::str_c("Downloading FASTQ from ENA: ", file_name))
    download.file(
      url      = url,
      destfile = dest_file,
      method   = "libcurl",
      mode     = "wb"
    )
  }
})

#Identify FASTQ files
seq_files <- fs::dir_ls(sample_dir, regexp = paste0(target_srr, ".*\\.fastq\\.gz$"))

# ## Run FastQC
# if (length(seq_files) > 0 && all(file.exists(seq_files))) {
#
#   # Verifica se o FastQC está instalado e acessível no ambiente
#   fastqc_path <- Sys.which("fastqc")
#
#   if (nzchar(fastqc_path)) {
#     message("Iniciando o controle de qualidade com FastQC...")
#
#     # Cria o diretório de saída caso ainda não exista
#     if (!dir.exists(qc_dir)) dir.create(qc_dir, recursive = TRUE)
#
#     system2(
#       command = "fastqc",
#       args    = c(
#         "--outdir", shQuote(qc_dir),
#         "--threads", "4",
#         purrr::map_chr(seq_files, shQuote)
#       )
#     )
#
#     message("FastQC concluído com sucesso!")
#   } else {
#     warning("O executável 'fastqc' não foi encontrado no PATH do sistema. Pulei esta etapa.")
#   }
# } else {
#   warning("Nenhum arquivo de sequenciamento válido foi encontrado em 'seq_files'. FastQC não executad

```

```
# }
```

Quebrar sequencia para rodar mais rápido

```
# 1. Definir o arquivo de entrada e de saída
fastq_gz_input <- fs::path(sample_dir, "SRR7945375_1.fastq.gz") # ou .fastq
fastq_gz_output <- fs::path(sample_dir, "SRR7945375_1_1pct.fastq.gz")

# 2. Contar quantas reads existem para calcular 10%
total_reads <- countFastq(fastq_gz_input)$records
n_1pct <- round(total_reads * 0.01)

# 3. Criar o amostrador
set.seed(100) # Garante reprodutibilidade
fs <- FastqSampler(fastq_gz_input, n = n_1pct)

# 4. Amostrar as reads e salvar o novo FASTQ
sampled_reads <- yield(fs)
# writeFastq(sampled_reads, fastq_gz_output, compress = TRUE)

close(fs)
cat("Total de reads originais:", total_reads,
    "\nTotal de reads salvas:", length(sampled_reads), "\n")
```

```
## Total de reads originais: 10303594
## Total de reads salvas: 103036
```

Analisar qualidade do FastQC

```
# Aggregate FastQC quality reports
qc <- fastqcr::qc_aggregate(qc.dir = qc_dir)

## Rows: 10 Columns: 3
## -- Column specification -----
## Delimiter: "\t"
## chr (3): status, module, sample
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 8 Columns: 2
## -- Column specification -----
## Delimiter: "\t"
## chr (2): Measure, Value
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 16 Columns: 2
## -- Column specification -----
## Delimiter: "\t"
```

```

## chr (1): Duplication Level
## dbl (1): Percentage of total
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 0 Columns: 2
## -- Column specification -----
## Delimiter: "\t"
## chr (2): Total Deduplicated Percentage, 77.32443029620714
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 10 Columns: 3
## -- Column specification -----
## Delimiter: "\t"
## chr (3): status, module, sample
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 8 Columns: 2
## -- Column specification -----
## Delimiter: "\t"
## chr (2): Measure, Value
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 16 Columns: 2
## -- Column specification -----
## Delimiter: "\t"
## chr (1): Duplication Level
## dbl (1): Percentage of total
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## Rows: 0 Columns: 2
## -- Column specification -----
## Delimiter: "\t"
## chr (2): Total Deduplicated Percentage, 48.00739516094603
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.

```

```

#Summary
summary(qc)

```

```

## # A tibble: 10 x 7
## # Groups:   module [10]
##   module          nb_samples nb_fail nb_pass nb_warn failed warned
##   <chr>          <dbl>    <dbl>    <dbl>    <dbl> <chr>  <chr>
## 1 Adapter Content      2      0      2      0 <NA>  <NA>
## 2 Basic Statistics      2      0      2      0 <NA>  <NA>
## 3 Overrepresented sequences  2      2      0      0 SRR79~ <NA>
## 4 Per base N content    2      0      2      0 <NA>  <NA>
## 5 Per base sequence content  2      2      0      0 SRR79~ <NA>

```

```
## 6 Per base sequence quality      2      0      2      0 <NA> <NA>
## 7 Per sequence GC content        2      0      0      2 <NA> SRR79~
## 8 Per sequence quality scores    2      0      2      0 <NA> <NA>
## 9 Sequence Duplication Levels    2      1      1      0 SRR79~ <NA>
## 10 Sequence Length Distribution  2      0      0      2 <NA> SRR79~
```

```
#General statistics
```

```
qc_stats(qc)
```

```
## # A tibble: 2 x 5
##   sample      pct.dup pct.gc tot.seq seq.length
##   <chr>      <dbl> <dbl> <chr>   <chr>
## 1 SRR7945375_1_1pct  22.7   57 103036 35-151
## 2 SRR7945375_1     52.0   56 10303594 35-151
```

```
#Locate target FastQC zip file directly on disk
```

```
qc_file_path <- fs::path(qc_dir, stringr::str_c(target_srr, "_1_fastqc.zip"))
```

```
# Import parsed FastQC metrics object from the local zip archive using qc_read
```

```
qc_data <- fastqcr::qc_read(qc_file_path)
```

```
## Reading: /home/wasim/Área de trabalho/Github/collabs/Colaborações/Angela Kaysel/data/SRR7945375/fastqcr
## Rows: 10 Columns: 3-- Column specification -----
## Delimiter: "\t"
## chr (3): status, module, sample
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 8 Columns: 2--
## Delimiter: "\t"
## chr (2): Measure, Value
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 38 Columns: 7--
## Delimiter: "\t"
## chr (1): Base
## dbl (6): Mean, Median, Lower Quartile, Upper Quartile, 10th Percentile, 90th...
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 34 Columns: 2--
## Delimiter: "\t"
## dbl (2): Quality, Count
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 38 Columns: 5--
## Delimiter: "\t"
## chr (1): Base
## dbl (4): G, A, T, C
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 101 Columns: 1--
## Delimiter: "\t"
## dbl (2): GC Content, Count
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 38 Columns: 2--
## Delimiter: "\t"
## chr (1): Base
## dbl (1): N-Count
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 24 Columns: 2--
```

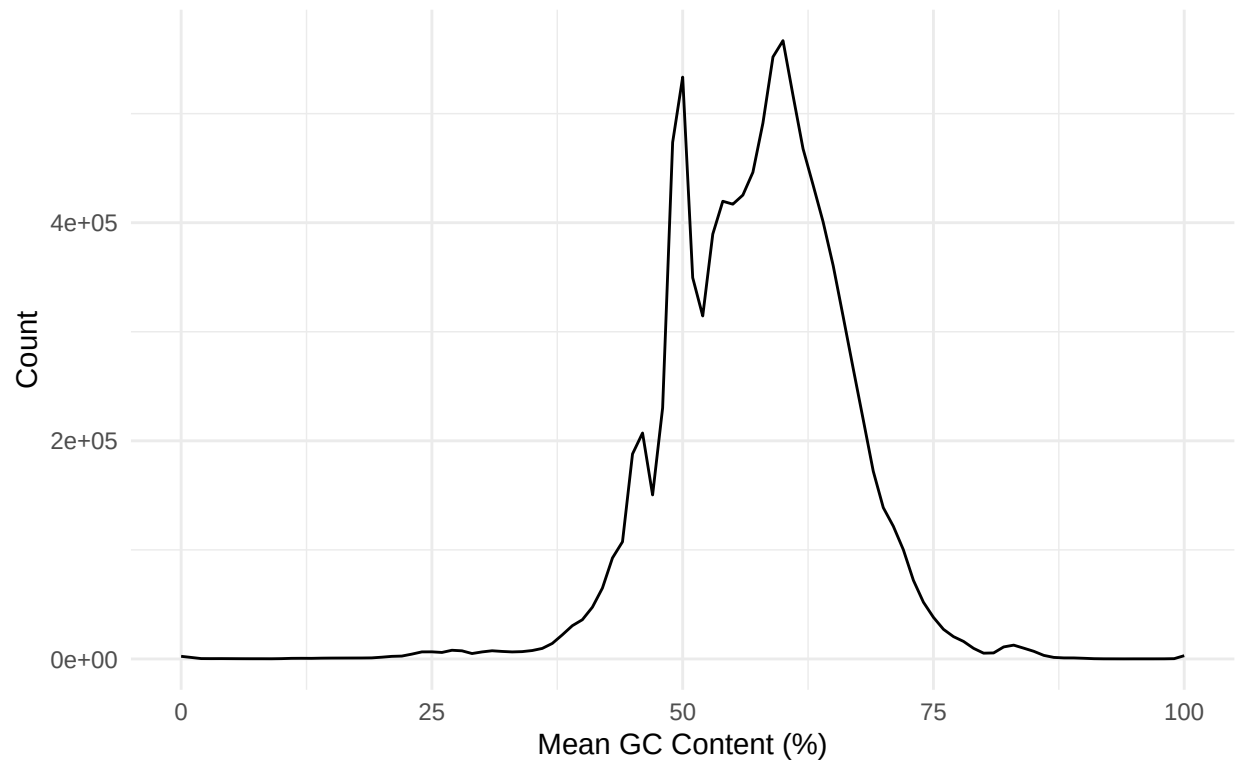
```
## Delimiter: "\t"
## chr (1): Length
## dbl (1): Count
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 16 Columns: 2-
## Delimiter: "\t"
## chr (1): Duplication Level
## dbl (1): Percentage of total
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 32 Columns: 4-
## Delimiter: "\t"
## chr (2): Sequence, Possible Source
## dbl (2): Count, Percentage
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 36 Columns: 7-
## Delimiter: "\t"
## chr (1): Position
## dbl (6): Illumina Universal Adapter, Illumina Small RNA 3' Adapter, Illumina...
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.Rows: 0 Columns: 2-
## Delimiter: "\t"
## chr (2): Total Deduplicated Percentage, 48.00739516094603
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# Save compiled quality control report
saveRDS(qc, file = fs::path(qc_dir, "fastqc_summary.rds"))

#Generate plots
qc_plot(qc_data, "Per sequence GC content")
```

```
## Warning: 'aes_string()' was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with 'aes()'.
## i See also 'vignette("ggplot2-in-packages")' for more information.
## i The deprecated feature was likely used in the fastqcr package.
## Please report the issue at <https://github.com/kassambara/fastqcr/issues>.
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```


Per sequence GC content

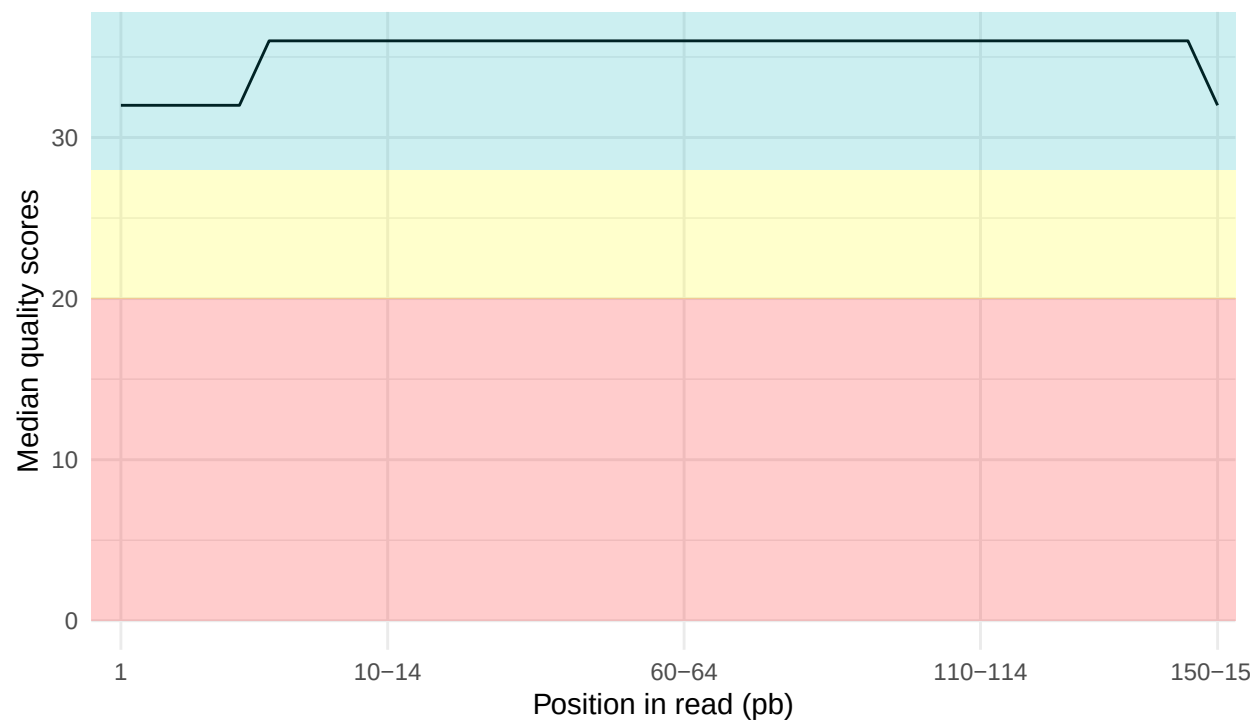


Status: WARN

```
qc_plot(qc_data, "Per base sequence quality")
```

Per base sequence quality

Red: low quality zone

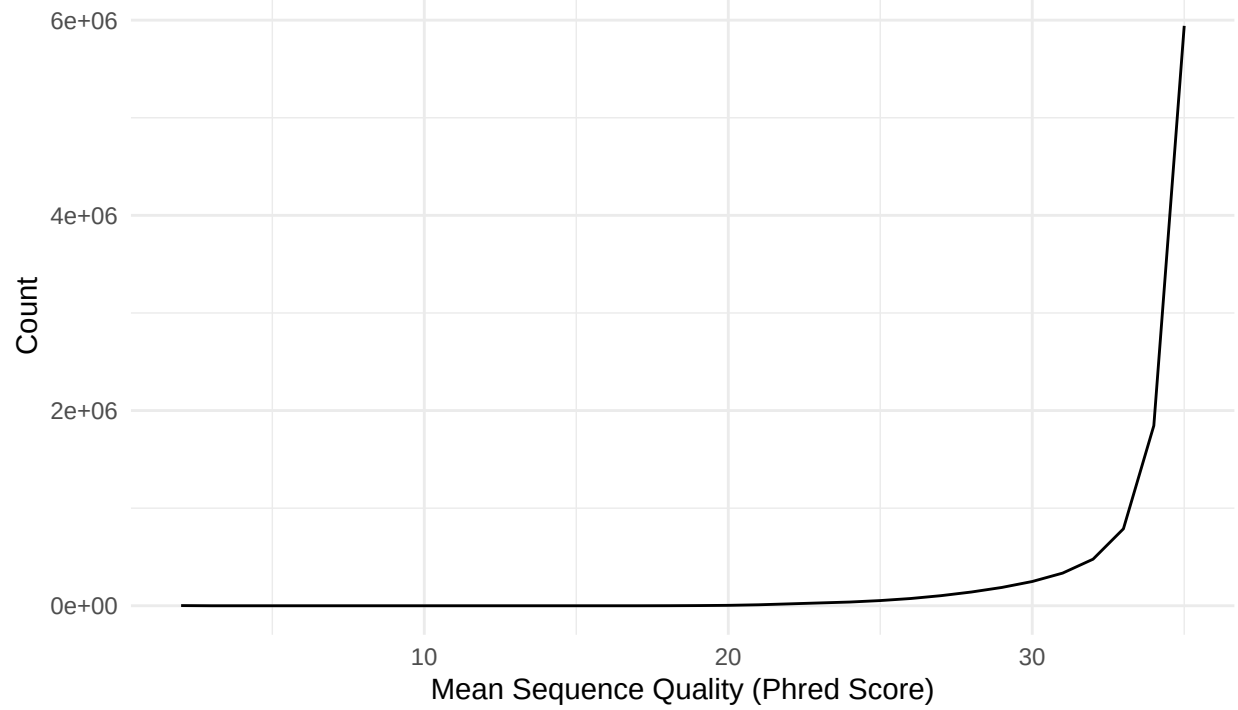


Status: PASS

```
qc_plot(qc_data, "Per sequence quality scores")
```

Per sequence quality scores

Quality score distribution over all sequences

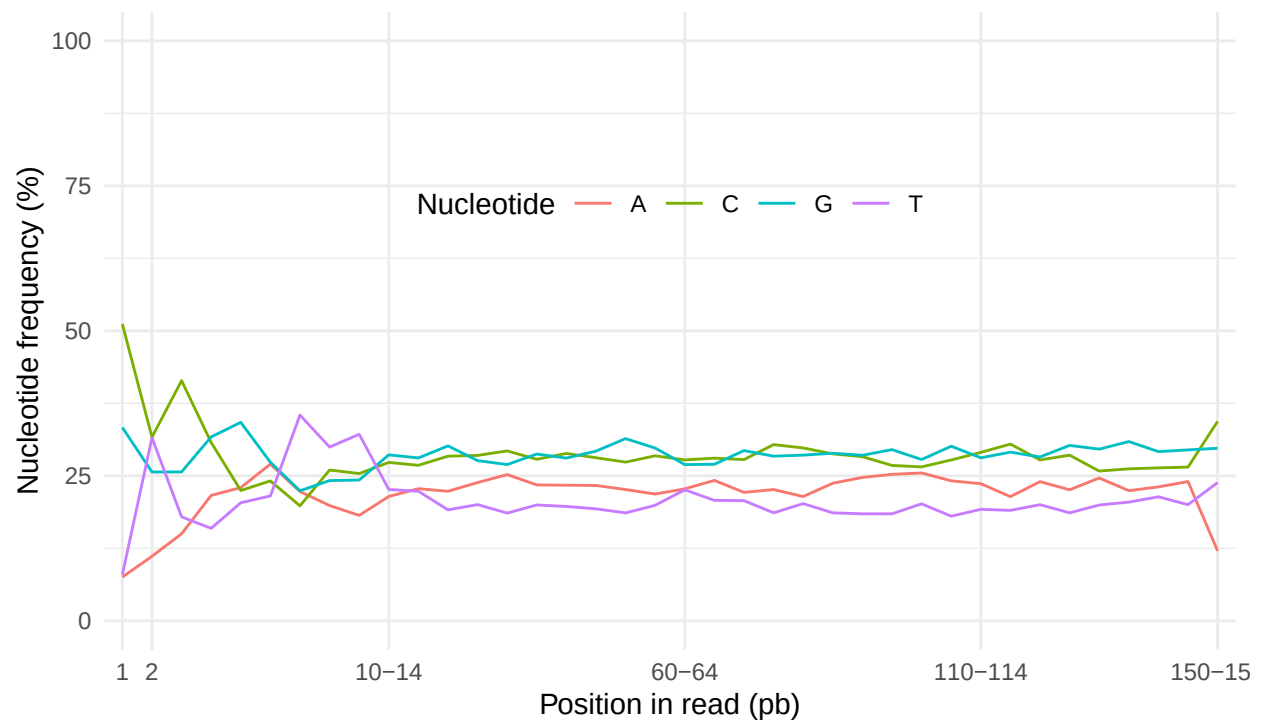


Status: PASS

```
qc_plot(qc_data, "Per base sequence content")
```

Per base sequence content

Sequence content across all bases

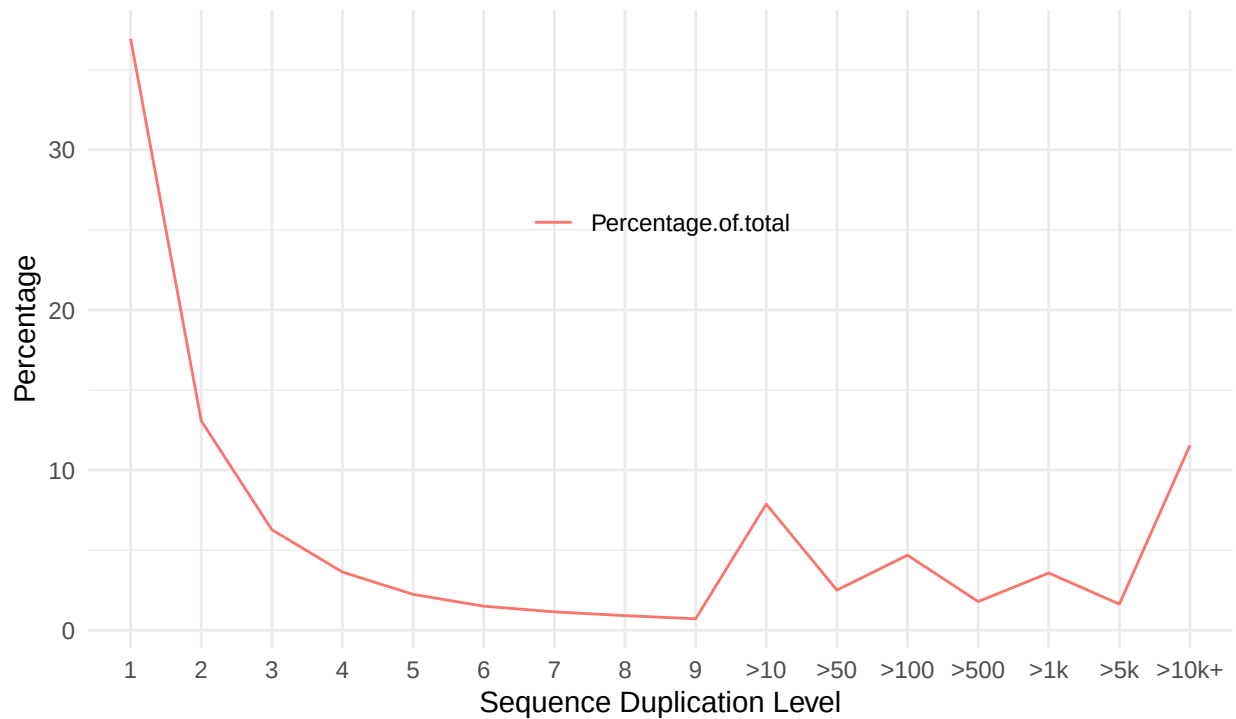


Status: FAIL

```
qc_plot(qc_data, "Sequence duplication levels")
```

Sequence Duplication Levels

Percentage of distinct reads: 48.01%



Status: FAIL

Predição via TrypncRNA

Baixar repositório

```
# 1. Diretórios principais e identificador da amostra
repo_path <- fs::path(project_dir, "TriTry-ncRNA")

# 2. Clonar o repositório GitHub caso ainda não exista localmente
if (!fs::dir_exists(repo_path)) {
  gert::git_clone(
    url = "https://github.com/AKCLAB/TriTry-ncRNA.git",
    path = repo_path
  )
}

# 3. Definição de pastas de entrada (data) e saída (results)
sample_dir <- fs::path(project_dir, "data", target_srr)
results_dir <- fs::path(project_dir, "results", target_srr)

# Garantir que os diretórios existem no disco
fs::dir_create(sample_dir)
fs::dir_create(results_dir)
```

```

# 4. Caminho exato do seu arquivo FASTQ (10% subamostrado)
target_fastq <- fs::path(sample_dir, "SRR7945375_1_1pct.fastq.gz")

# Verificar se o arquivo realmente existe antes de rodar
if (!fs::file_exists(target_fastq)) {
  warning("ATENÇÃO: O arquivo ", target_fastq, " não foi encontrado no disco!")
}

# 5. Escrever o caminho no arquivo listdir.txt exigido pelo pipeline
list_dir_file <- fs::path(sample_dir, "listdir.txt")
readr::write_lines(as.character(target_fastq), file = list_dir_file)

# 6. Definir caminhos das referências (arquivos de teste do repositório)
script_path <- fs::path(repo_path, "scripts", "pipe_ncrna.sh")
db_dir <- fs::path(repo_path, "database")
ref_fasta <- fs::path(repo_path, "test", "LBRAZ_M2903.Dec2022.fasta")
ref_gff <- fs::path(repo_path, "test", "LBRAZ_M2903.Dec2022.gff")
utr5_file <- fs::path(repo_path, "test", "UTRme_fiveutr.tsv")
utr3_file <- fs::path(repo_path, "test", "UTRme_threutr.tsv")

# # 7. Executar o pipeline Tryp-ncRNA via system2
# system2(
#   command = "bash",
#   args = c(
#     shQuote(script_path),
#     "-dir_list", shQuote(list_dir_file),
#     "-output", shQuote(results_dir),
#     "-db", shQuote(db_dir),
#     "-threads", "12",
#     "-reffasta", shQuote(ref_fasta),
#     "-refgff", shQuote(ref_gff),
#     "-utr5", shQuote(utr5_file),
#     "-utr3", shQuote(utr3_file),
#     "-dir_tool", shQuote(repo_path)
#   )
# )

```

Rodar no Docker

```

## Rodar no terminal =====
# #Install docker
# sudo snap install docker
#
# # 1. Fazer o Build da imagem
# sudo docker build -t tryp -f TriTry-ncRNA/docker/Dockerfile .
#
# # 2. Iniciar o container montando a pasta atual
# sudo docker run -v `pwd`:./workspace -w /workspace -it tryp
#
# # 3 e 4. Rodar o script direto através do ambiente conda criado no Dockerfile
# conda run -n tryp-ncrna bash /workspace/TriTry-ncRNA/scripts/pipe_ncrna.sh \
#   -dir_list /workspace/data/SRR7945375/listdir.txt \

```

```
# -output /workspace/results/SRR7945375 \
# -db /workspace/TriTry-ncRNA/database \
# -threads 8 \
# -reffasta /workspace/TriTry-ncRNA/test/LBRAZ_M2903.Dec2022.fasta \
# -refgff /workspace/TriTry-ncRNA/test/LBRAZ_M2903.Dec2022.gff \
# -utr5 /workspace/TriTry-ncRNA/test/UTRme_fiveutr.tsv \
# -utr3 /workspace/TriTry-ncRNA/test/UTRme_threeutr.tsv \
# -dir_tool /programs
```

Analisar resultados

Limpar dados

```
# Importar a tabela final de caracterização gerada pelo pipeline
ncrna_raw <- read_tsv(
  here(project_dir, "results", "LbraziliensisMHOMBR75M2903_merged_processed.bed"),
  col_names = c("chrom", "start", "end", "name", "score", "strand"),
  show_col_types = FALSE
)

# Tratar e classificar os ncRNAs
ncrna_classified = ncrna_raw %>%
  #Limpar tabela
  #Retirar cromossomos scaffold (?)
  filter(!str_detect(chrom, "SCAFO")) %>%
  #Name = ncRNA name + classe
  separate(name, into = c("name", "class"), sep = "_") %>%
  #Chrom = species + chrom/scaffold
  separate(chrom, into = c("species", "chrom"), sep = "_") %>%
  #Limpar nome da espécie
  mutate(species = if_else(species == "LBRM2903", "LbrM2903", species)) %>%
  #Calcular tamanho
  mutate(size = end-start) %>%
  #Classificar por tamanho e função
  mutate(
    group = case_when(
      class %in% c("SL-RNA", "snRNA", "IRES") ~ "Splicing\n& Processing",
      class %in% c("tRNA", "rRNA", "SRP") ~ "Translation\n& Ribosome",
      class %in% c("gRNA", "snoRNA", "scarRNA") ~ "RNA Modification\n& Editing",
      class %in% c("lncRNA") ~ "Long ncRNA",
      class %in% c("miRNA", "sRNA", "Cis-reg") ~ "Small Regulatory\n& Cis-elements",
      class %in% c("no-RNA", "Unclassified") ~ "Unannotated\n& Non-ncRNA",
      TRUE ~ "Other"
    )
  )

#Print
ncrna_classified
```

```
## # A tibble: 20,204 x 10
```

```
##   species  chrom start   end name      class      score strand  size group
```

```
##      <chr>      <chr> <dbl> <dbl> <chr>      <chr>      <dbl> <chr>      <dbl> <chr>
## 1 LbrM2903 01      8349 8384 ncRNA00234 sRNA      255 -      35 "Small~
## 2 LbrM2903 01      8350 8384 ncRNA00235 sRNA      255 +      34 "Small~
## 3 LbrM2903 01      21952 21992 ncRNA00236 sRNA      255 +      40 "Small~
## 4 LbrM2903 01      21952 21992 ncRNA00237 Unclassified 255 -      40 "Unann~
## 5 LbrM2903 01      26147 26168 ncRNA00238 sRNA      255 +      21 "Small~
## 6 LbrM2903 01      26147 26168 ncRNA00239 sRNA      255 -      21 "Small~
## 7 LbrM2903 01      33133 33245 ncRNA00240 miRNA      42.5 -      112 "Small~
## 8 LbrM2903 01      33133 33244 ncRNA00241 miRNA      85 +      111 "Small~
## 9 LbrM2903 01      35331 35366 ncRNA00242 sRNA      255 +      35 "Small~
## 10 LbrM2903 01      35332 35364 ncRNA00243 sRNA      255 -      32 "Small~
## # i 20,194 more rows
```

```
#Calcular estadísticas
ncrna_classified_summary = ncrna_classified %>%
  group_by(group, class) %>%
  summarise(median_size = median(size, na.rm = T),
            mean_size = mean(size, na.rm = T),
            n_ncrnas = n(),
            sd_size = sd(size, na.rm = T),
            se_size = sd_size/sqrt(n_ncrnas)) %>%
  ungroup()
```

```
## 'summarise()' has regrouped the output.
## i Summaries were computed grouped by group and class.
## i Output is grouped by group.
## i Use 'summarise(.groups = "drop_last")' to silence this message.
## i Use 'summarise(.by = c(group, class))' for per-operation grouping
## ('?dplyr::dplyr_by') instead.
```

```
#Print
ncrna_classified_summary
```

```
## # A tibble: 14 x 7
##   group      class median_size mean_size n_ncrnas sd_size se_size
##   <chr>      <chr>      <dbl>      <dbl>      <int>      <dbl>      <dbl>
## 1 "Long ncRNA" lncR~      693      807.        29      670.      124.
## 2 "RNA Modification\n& Ed~ scaR~      291      291          1      NA      NA
## 3 "RNA Modification\n& Ed~ snoR~      80      93.0        269      81.2      4.95
## 4 "Small Regulatory\n& Ci~ Cis~      156.      201.         26      117.      22.9
## 5 "Small Regulatory\n& Ci~ miRNA      95      104.         248      37.8      2.40
## 6 "Small Regulatory\n& Ci~ sRNA      32      36.3      15229      20.2      0.164
## 7 "Splicing\n& Processing" IRES      94      97.9          7      30.9      11.7
## 8 "Splicing\n& Processing" SL-R~      193      194.          11      11.0      3.31
## 9 "Splicing\n& Processing" snRNA      117      118.          8      36.3      12.8
## 10 "Translation\n& Ribosom~ SRP      278      278          2          0          0
## 11 "Translation\n& Ribosom~ rRNA      22      35.0         723      178.      6.62
## 12 "Translation\n& Ribosom~ tRNA      73      68.9         204      77.2      5.40
## 13 "Unannotated\nNon-ncRNA" Uncl~      23      26.1      3446      9.55      0.163
## 14 "Unannotated\nNon-ncRNA" no-R~      1335      1335          1      NA      NA
```


Distribuições

```
#Em quais cromossomos estão esses ncRNAs?
ncrna_classified_chromosome = ncrna_classified %>%
  group_by(chrom, group) %>%
  summarise(n = n()) %>%
  ungroup() %>%
  ggplot() +
  aes(y = fct_rev(chrom),
      x = n,
      fill = group) +
  labs(
    title = "Distribution of ncRNA groups, by Chromosome",
    y = "Chromosome",
    x = "Total",
    fill = "Group"
  ) +
  facet_grid(~group, scales = "free") +
  geom_col(color = "white",
           size = 0.2) +
  theme_classic() +
  scale_x_continuous(expand = c(0,0)) +
  ggsci::scale_fill_npg() +
  theme(
    axis.title = element_text(size = 10, face = "bold"),
    axis.text = element_text(size = 10),
    axis.ticks.x = element_line(linewidth = 1),
    axis.ticks.y = element_line(linewidth = 1),
    legend.position = "top",
    legend.key.width = unit(0.2, "cm"),
    legend.key.height = unit(0.5, "cm"),
    legend.text = element_text(size = 10),
    strip.text = element_text(size = 10, color = "black", face = "bold"),
    strip.background = element_blank(),
    panel.grid.major.x = element_line(color = "gray90", linetype = 2),
    plot.title = element_text(face = "bold", size = 12, hjust = 0.5),
    plot.title.position = "panel",
  ) +
  guides(fill = guide_legend(nrow = 1))
```

```
## 'summarise()' has regrouped the output.
## i Summaries were computed grouped by chrom and group.
## i Output is grouped by chrom.
## i Use 'summarise(groups = "drop_last")' to silence this message.
## i Use 'summarise(by = c(chrom, group))' for per-operation grouping
## ('?dplyr::dplyr_by') instead.
```

```
## Warning in geom_col(color = "white", size = 0.2): Ignoring unknown parameters:
## 'size'
```

```

#Qual o tamanho dessas classes, por grupo? Boxplot + Barras
ncrna_classified_summary_bars = ncrna_classified_summary %>%
  arrange(median_size) %>%
  mutate(class = fct_inorder(class)) %>%
  ggplot() +
  aes(x = median_size,
       y = class) +
  labs(title = "ncRNA class, by median size",
       subtitle = "Bars correspond to the median, with boxplots showing the IQR.",
       x = "Size (nt)",
       y = "Class",
       fill = "Group",
       ) +
  geom_col(aes(fill = group)) +
  geom_boxplot(data = ncrna_classified,
               aes(x = size),
               width = 0.5,
               alpha = 0, outliers = F) +
  theme_classic() +
  scale_x_continuous(expand = c(0,0)) +
  ggsci::scale_fill_npg() +
  theme(
    axis.title = element_text(size = 10, face = "bold"),
    axis.text = element_text(size = 10),
    legend.position = "top",
    legend.key.width = unit(0.2, "cm"),
    legend.key.height = unit(0.5, "cm"),
    legend.text = element_text(size = 10),
    panel.grid.major.x = element_line(color = "gray90", linetype = 2),
    plot.title = element_text(face = "bold", size = 12, hjust = 0.5),
    plot.title.position = "plot") +
  guides(fill = guide_legend(nrow = 1))

```

```

#Qual a distribuição dos tamanhos? Visualizar pontos
ncrna_classified_distribution = ncrna_classified %>%
  mutate(class = fct_reorder(class, size, .fun = median)) %>%
  ggplot(aes(x = class, y = size, fill = group, color = group)) +

  stat_halfeye(
    side = "right",
    adjust = 0.5,
    width = 0.6,
    .width = 0,
    justification = -0.2,
    point_colour = NA,
    alpha = 0.7
  ) +

  geom_boxplot(
    width = 0.15,
    outlier.shape = NA,
    alpha = 1,

```

```

    color = "black"
  ) +

  stat_dots(
    side = "left",
    justification = 1.2,
    binwidth = NA,
    dotsize = 0.4,
    alpha = 0.3
  ) +

  facet_wrap(~group, scales = "free",
             ncol = 1) +

  labs(
    title = "Distribution of ncRNA size,\nby class and group",
    y = "Size (nt)",
    x = "Class",
    fill = "Group"
  ) +

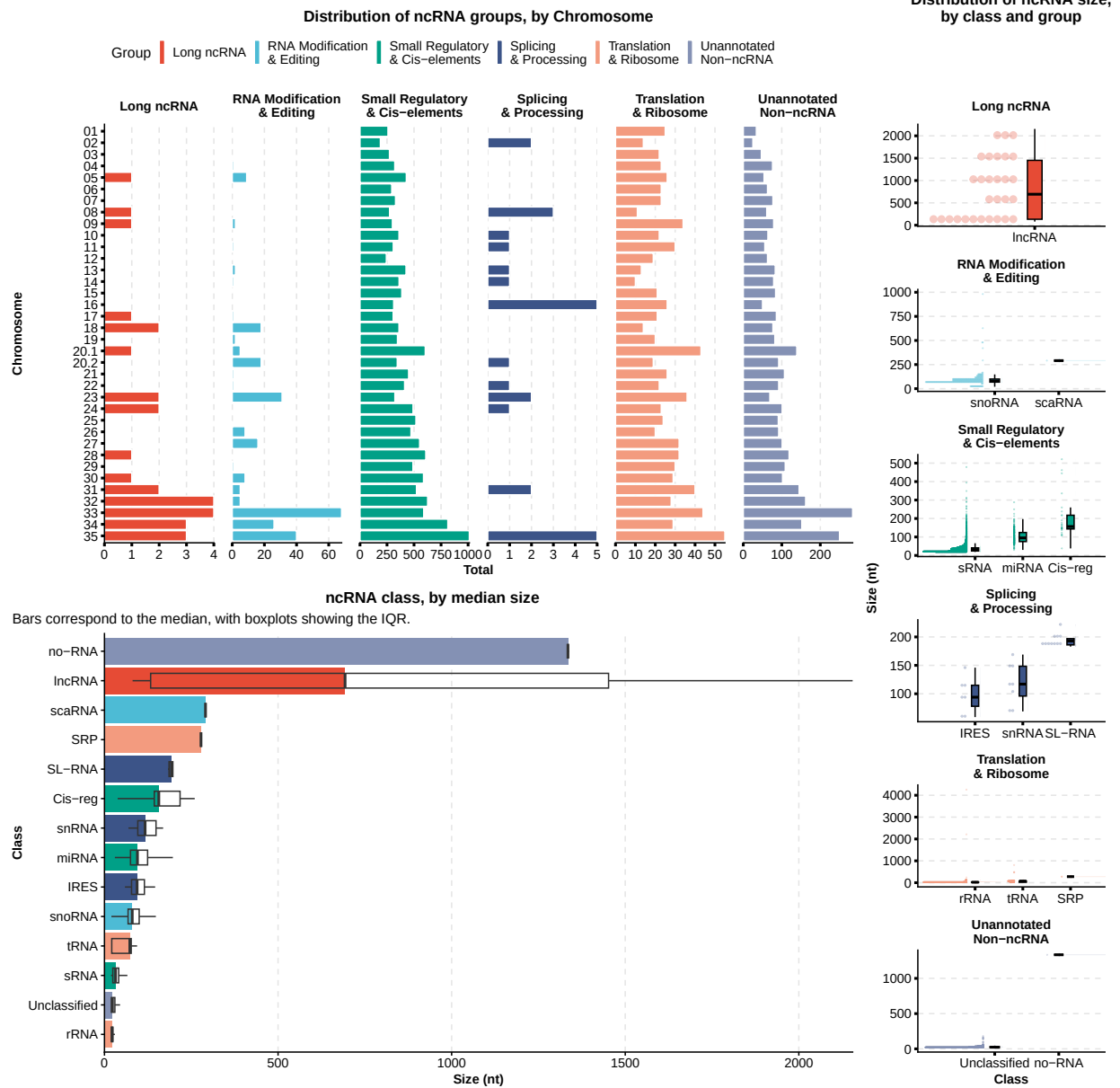
  theme_classic() +
  # scale_y_continuous(expand = c(0,0)) +
  ggsci::scale_fill_npg() +
  ggsci::scale_color_npg() +
  theme(
    axis.title = element_text(size = 10, face = "bold"),
    axis.text = element_text(size = 10),
    axis.ticks.x = element_line(linewidth = 1),
    axis.ticks.y = element_line(linewidth = 1),
    legend.position = "none",
    strip.text = element_text(size = 10, color = "black", face = "bold"),
    strip.background = element_blank(),
    panel.grid.major.y = element_line(color = "gray90", linetype = 2),
    plot.title = element_text(face = "bold", size = 12, hjust = 0.5),
    plot.title.position = "panel"
  )

#Merge plots
a = ncrna_classified_chromosome + theme(panel.spacing.x = unit(0.5, "cm"))
b = ncrna_classified_summary_bars + theme(legend.position = "none")
c = ncrna_classified_distribution

ncrna_classified_patch =
  ((a/b) | (c)) +
  plot_layout(guides = "collect",
              widths = c(0.8, 0.2),
              heights = c(0.8, 0.2)) +
  plot_annotation(theme = theme(legend.position = "top",
                                legend.justification = "left"))

#Print
ncrna_classified_patch

```



```
#Save
ncrna_classified_patch %>%
  ggsave(filename = here(project_dir, "Figures", "ncRNA_summaries_bars_distribution.png"),
    width = 12,
    height = 12)
```